# PASSWORD STRENGTH CHECKER

A Beginner-Friendly Cybersecurity Mini Project Using Python

| | |
|---|---|
| **Project Domain** | Cybersecurity |
| **Programming Language** | Python 3 |
| **Interface** | Command-line interface |
| **Libraries** | re and getpass (built-in) |

## Project overview

This project checks a password against five basic characteristics: minimum length, uppercase letters, lowercase letters, numbers, and special characters. It assigns a score and labels the result as Weak, Medium, or Strong, then gives suggestions for improvement.

Prepared as a simple college-level cybersecurity project. You can add your name, department, and college details before submission.

## 1. Abstract

The Password Strength Checker is a cybersecurity mini project that helps users understand whether a password meets basic strength rules. It checks the password length and character types, calculates a score out of five, and displays improvement suggestions. The project introduces password security concepts in a simple and practical way.

## 2. Objectives

• Identify passwords that do not meet basic strength checks.

• Check length, uppercase and lowercase letters, digits, and special characters.

• Provide suggestions to help users improve password quality.

• Promote awareness of secure password practices.

## 3. Software Requirements

| Requirement | Details |
|---|---|
| Programming language | Python 3 |
| IDE | VS Code, IDLE, or PyCharm |
| Operating system | Windows or Linux |
| Libraries | re and getpass (included with Python) |

## 4. Algorithm

1. Start the program.

2. Read a password from the user.

3. Check whether it has at least 8 characters.

4. Check for uppercase letters, lowercase letters, digits, and special characters.

5. Add one point for each rule that is satisfied.

6. Display Weak, Medium, or Strong based on the score.

7. Show suggestions for any missing rule.

8. Stop the program.

## 5. Source Code

```
import re
import getpass

def check_password(password):
    score = 0
    suggestions = []

    if len(password) >= 8:
        score += 1
    else:
        suggestions.append("Use at least 8 characters.")

    if re.search(r"[A-Z]", password):
        score += 1
    else:
        suggestions.append("Add an uppercase letter.")

    if re.search(r"[a-z]", password):
```

```
        score += 1
    else:
        suggestions.append("Add a lowercase letter.")

    if re.search(r"[0-9]", password):
        score += 1
    else:
        suggestions.append("Add a number.")

    if re.search(r"[^A-Za-z0-9]", password):
        score += 1
    else:
        suggestions.append("Add a special character.")

    if score <= 2:
        strength = "WEAK"
    elif score <= 4:
        strength = "MEDIUM"
    else:
        strength = "STRONG"

    print("\\nPassword Strength:", strength)
    print("Score:", score, "/ 5")

    if suggestions:
        print("\\nSuggestions:")
        for item in suggestions:
            print("-", item)
    else:
        print("Good! Your password meets all five checks.")

print("PASSWORD STRENGTH CHECKER")
password = getpass.getpass("Enter a password: ")
check_password(password)
```

## 6. Sample Output

**Example 1: Weak password**

```
PASSWORD STRENGTH CHECKER
Enter a password: [hidden input]

Password Strength: WEAK
Score: 2 / 5

Suggestions:
- Use at least 8 characters.
- Add an uppercase letter.
- Add a special character.
```

**Example 2: Strong password**

```
PASSWORD STRENGTH CHECKER
Enter a password: [hidden input]

Password Strength: STRONG
Score: 5 / 5
Good! Your password meets all five checks.
```

## 7. Working Principle

The program uses Python regular expressions to search for different character types. Each satisfied rule adds one point. A score of 0–2 is classified as Weak, 3–4 as Medium, and 5 as Strong. Suggestions are shown for rules that are not satisfied.

## 8. Advantages

• Easy to understand and implement.

• Uses only Python's built-in libraries.

• Provides immediate feedback and suggestions.

• Can be extended with a graphical or web interface.

## 9. Future Enhancements

• Build a graphical interface using Tkinter.

• Create a web version using Flask or Django.

• Add a password generator.

• Check for common passwords using a safe, privacy-preserving approach.

• Add a visual strength meter.

## 10. Conclusion

The Password Strength Checker is a beginner-friendly cybersecurity project developed in Python. It checks basic password characteristics, identifies missing elements, and offers suggestions to encourage safer password creation. It is an educational tool and does not replace a full password-security assessment.

## 11. Security Note

Do not enter your real account password into a classroom demo or store passwords in plain text. This checker only evaluates a few simple rules; password length, uniqueness, and resistance to guessing also matter. Passing all five checks does not automatically mean a password is secure.